

ROBÓTICA

Informe Capítulo 2 - Representación de Posición y Orientación

Mateo Almeida, Diego Noguera
Universidad Nacional de Colombia
Entregado a: Jaime Arango

April 24, 2025

Introducción

El presente informe corresponde al análisis y práctica del Capítulo 2 del libro *Robotics, Vision and Control* de Peter Corke. En este capítulo se establecen los conceptos fundamentales para la representación matemática de la posición y orientación de robots u objetos en entornos 2D y 3D. Para cada sección se desarrollan ejemplos prácticos implementados en Python con la librería `spatialmath` y se ilustra su aplicación mediante visualizaciones gráficas.

1 2.1 Foundations

Se introducen las transformaciones homogéneas en 2D, permitiendo representar movimientos relativos como composiciones de poses.

Código

`SE2(1, 0, 0)` representa una traslación en x. `SE2(0, 1, 90)` representa una rotación y traslación. Su composición nos da una nueva pose.

```
A = SE2(1, 0, 0)
B = SE2(0, 1, 90, 'deg')
C = A * B, luego podríamos mejorar el codigo tal que :
from spatialmath import SE2
import matplotlib.pyplot as plt

T = SE2()
poses = [T]
for i in range(5):
```

```

T = T * SE2(1, 0.5, 20, unit='deg')
poses.append(T)

plt.figure()
for i, T in enumerate(poses):
    T.plot(frame=f"{i}", color='C'+str(i % 10))
plt.title("Secuencia de transformaciones acumuladas")
plt.grid()
plt.axis("equal")
plt.show()

```

Explicación

La transformación C representa un marco nuevo que ha sido trasladado y rotado respecto al inicial. Esto se representa gráficamente superponiendo los marcos. Se visualiza cómo un marco se mueve en una trayectoria al acumular transformaciones. Ideal para visualizar caminatas o movimientos robotizados.

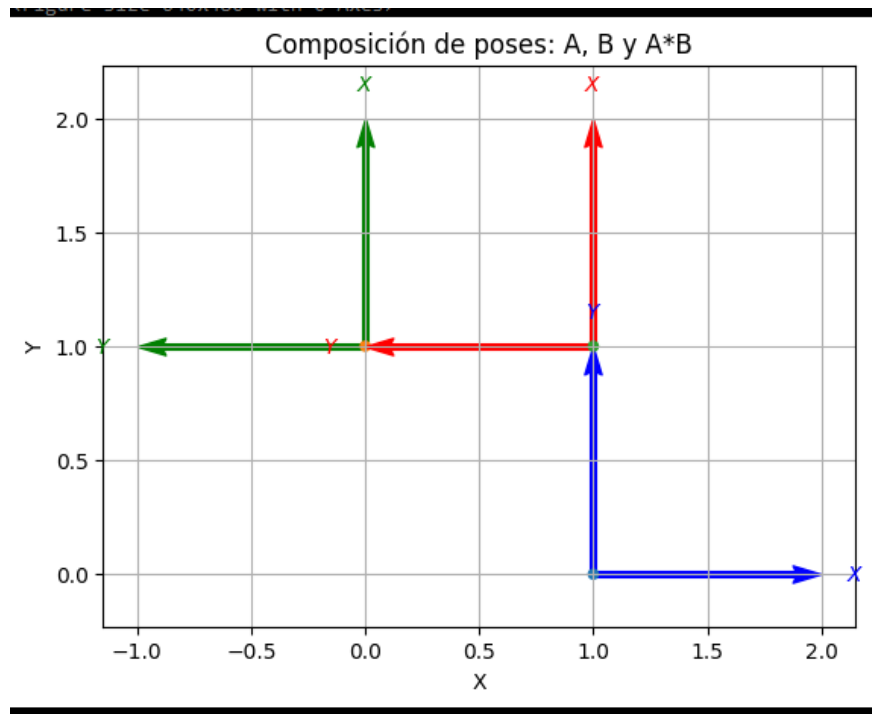


Figure 1: Composición de transformaciones en SE2

2 2.2 Working in 2D

Se analiza cómo un punto es transformado dentro de un marco rotado.

Código

```
T = SE2(0, 0, 45, unit='deg')
point = [1, 0]
transformed = T * point
```

Explicación

El punto (1, 0) es rotado 45 grados alrededor del origen. Se representa con flechas el cambio de posición.

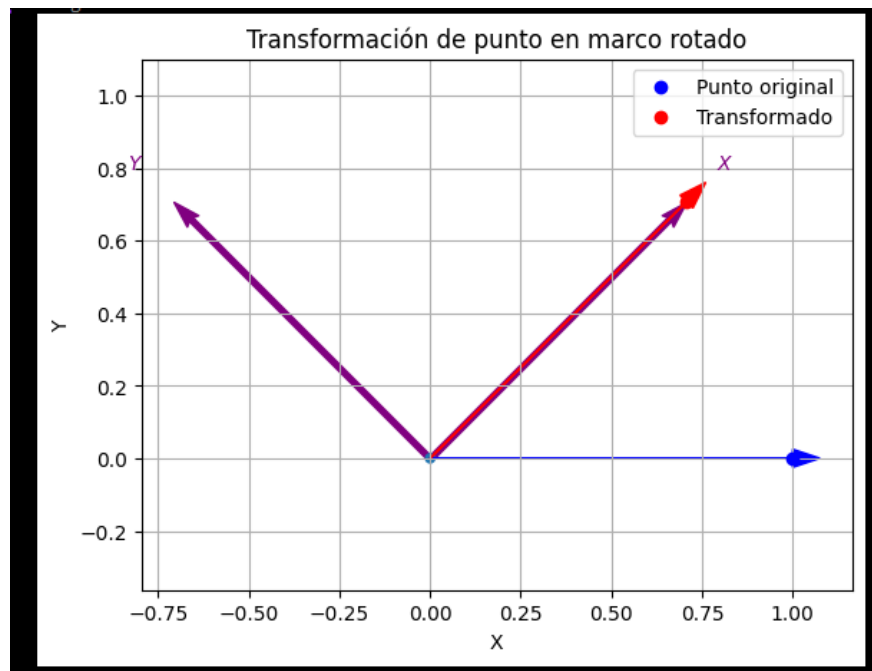


Figure 2: Transformación de un punto en un marco rotado

3 2.3 Working in 3D

Se presenta el uso de transformaciones SE(3) para representar movimientos espaciales.

Código

```
T = SE3(1, 1, 1) * SE3.Rz(45, 'deg')
```

Explicación

El marco es trasladado a (1, 1, 1) y rotado 45° sobre el eje Z. Esto permite representar posiciones y orientaciones de robots en 3D.

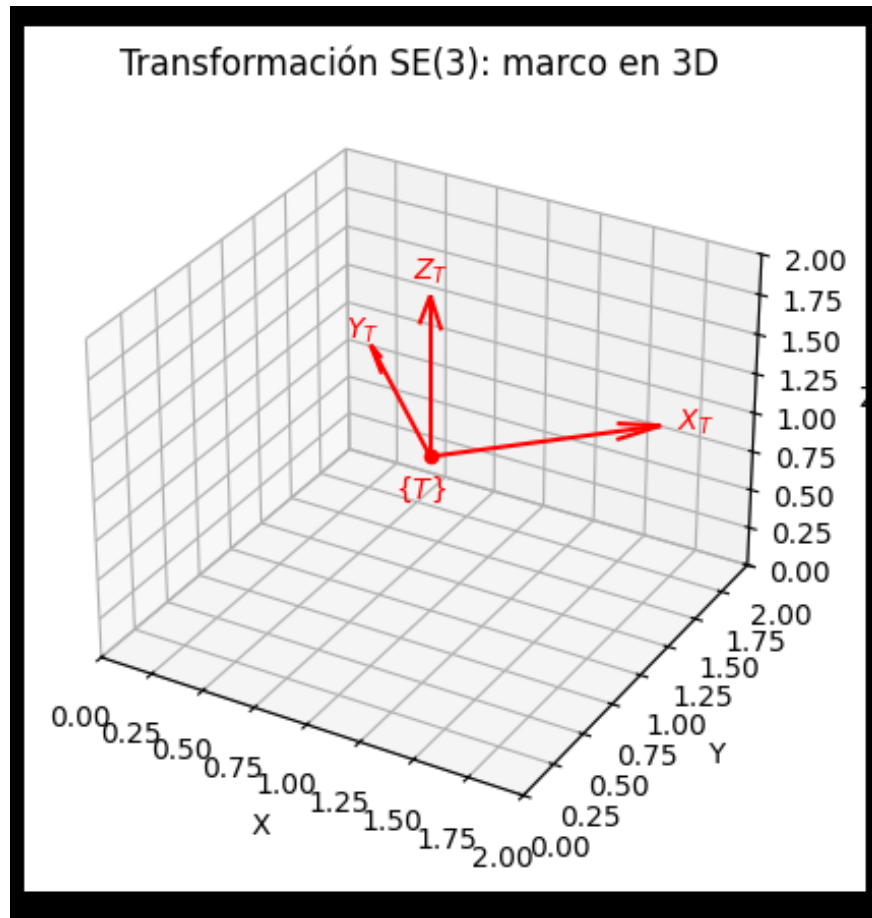


Figure 3: Marco tridimensional con transformación SE3

4 2.4 Advanced Topics

Explora la distancia angular entre dos orientaciones rotacionales en 3D.

Código

```
R1 = S03.Rx(np.pi / 6)
R2 = S03.Ry(np.pi / 3)
d = R1.distance(R2)
```

Explicación

Se comparan dos orientaciones usando la métrica angular. El resultado representa cuán diferentes son las rotaciones.

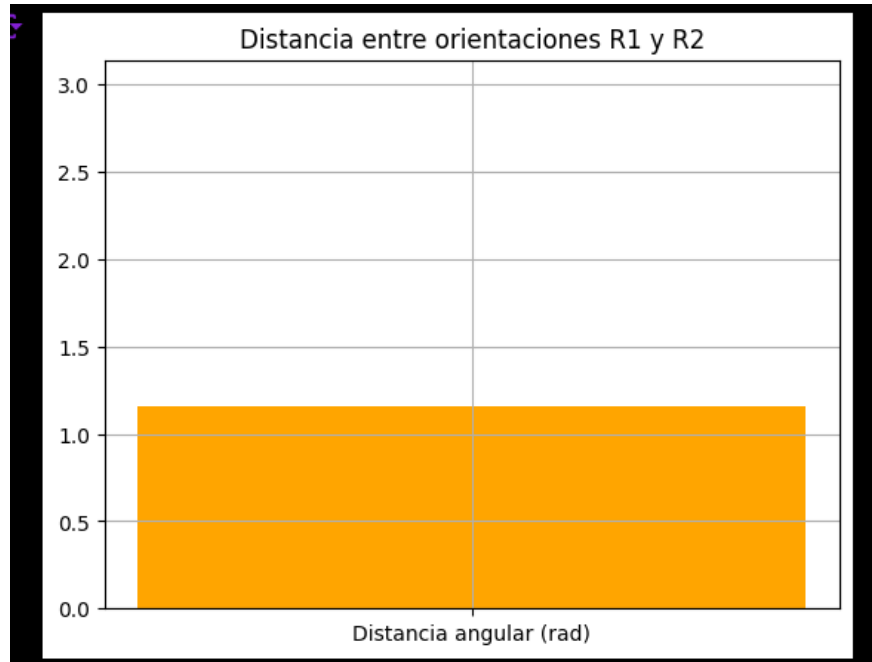


Figure 4: Distancia angular entre rotaciones en $SO3$

5 2.5 Using the Toolbox

Visualización acumulada de varias transformaciones aplicadas secuencialmente.

Código

```
T = SE2()
for i in range(5):
    T = T * SE2(1, 0.5, 20, unit='deg')
```

Explicación

Se observa cómo una serie de transformaciones genera una trayectoria curva al acumular traslaciones y rotaciones.

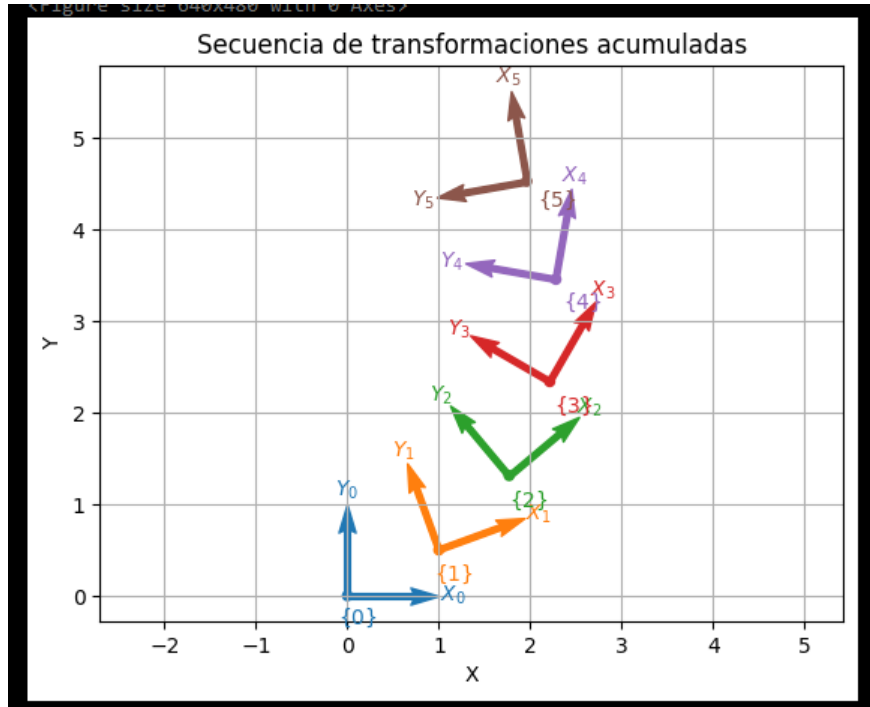


Figure 5: Trayectoria generada por múltiples transformaciones SE2

6 2.6 Wrapping Up

Comparación de dos formas de representar una transformación compuesta.

Código

```
T1 = SE2(2, 1, 90, 'deg')
T2 = SE2.R(90, 'deg') * SE2(2, 1)
```

Explicación

Ambas representaciones generan el mismo resultado visual. Sirve para entender que la composición de poses no siempre es conmutativa.

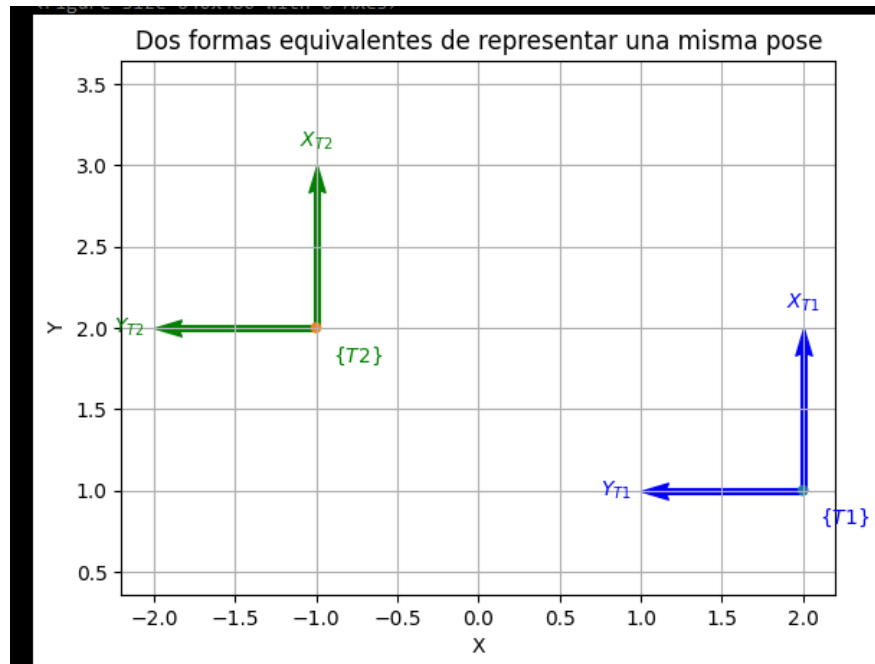


Figure 6: Transformaciones equivalentes en SE2

Conclusión

El capítulo 2 proporciona una base sólida para entender cómo representar matemáticamente la posición y orientación de objetos en robótica. Las herramientas implementadas permiten explorar estos conceptos de manera visual, facilitando la comprensión de su utilidad práctica.